Computer Exercise 2: Worked solutions

Algorithms, Programming, and Data Representation · Module 92313 · Exercise 2

From Fundamentals to AI Apocalypse · personal study guide

Contents

Task 1: IEEE 754 bit fields 2

Task 2: Classify IEEE 754 values 2

Task 3: Count representable floats between two values 2

Task 4: Compare ordinary and compensated sums 2

Task 5: Implement Kahan compensated summation 3

Task 6: Detect likely catastrophic cancellation 3

Task 7: Compare floats with a relative tolerance 3

Task 8: Represent money as integer cents 3

Task 9: Audit cumulative transaction drift 4

Task 10: Generate a summation stress case 4

Answers 5

Made by Thanate Amornjiraporn (Focus)

This exercise studies binary64 values and rounding drift in financial sums. Try each task before opening its worked steps. For the personalized transaction analysis, calculate and report the values from your own assigned dataset.

Task 1: IEEE 754 bit fields

Connected topic: IEEE 754 formats (§4.4)

Task 01 · to_ieee754_bits(x)

Try first: Start by writing the 64-bit layout and the bias. Then encode one exactly representable value by hand before checking your output.

Answers: see "Task 1: IEEE 754 bit fields" under Answers at the end.

Task 2: Classify IEEE 754 values

Connected topic: Special values (§4.5)

Task 02 · classify_float(x)

Try first: Build a four-case decision table from the exponent and fraction fields; test signed zero and each special-value class.

Answers: see "Task 2: Classify IEEE 754 values" under Answers at the end.

Task 3: Count representable floats between two values

Connected topic: IEEE 754 formats (§4.4)

Task 03 · ulp_distance(a, b)

Try first: Map the 64-bit patterns into an integer ordering that increases with numeric value, then compare the mapped integers.

Answers: see "Task 3: Count representable floats between two values" under Answers at the end.

Task 4: Compare ordinary and compensated sums

Connected topic: Floating-point arithmetic (§4.3)

Task 04 · naive_sum(values)

Try first: Use the same input order for both algorithms and explain why each addition rounds.

Answers: see "Task 4: Compare ordinary and compensated sums" under Answers at the end.

Made by Thanate Anonjiraporn (Focus)

Task 5: Implement Kahan compensated summation

Connected topic: Floating-point arithmetic (§4.3)

Task 05 · kahan_sum(values)

Try first: Track the rounding loss in a correction variable and subtract it from the next input.

Answers: see "Task 5: Implement Kahan compensated summation" under Answers at the end.

Task 6: Detect likely catastrophic cancellation

Connected topic: Floating-point arithmetic (§4.3)

Task 06 · detect_catastrophic_cancellation(a, b)

Try first: Compare the subtraction result with the magnitude of its operands, while handling nonfinite values first.

Answers: see "Task 6: Detect likely catastrophic cancellation" under Answers at the end.

Task 7: Compare floats with a relative tolerance

Connected topic: Floating-point arithmetic (§4.3)

Task 07 · safe_float_equal(a, b, tol=1e-9)

Try first: Handle NaN and infinities explicitly, then compare the error to a scale based on the operands.

Answers: see "Task 7: Compare floats with a relative tolerance" under Answers at the end.

Task 8: Represent money as integer cents

Connected topic: Floating-point arithmetic (§4.3)

Task 08 · FixedPointCents

Try first: Store a signed integer cent count and implement the required arithmetic and representation methods.

Answers: see "Task 8: Represent money as integer cents" under Answers at the end.

Made by Thanate Amornjiraporn (Focus)

Task 9: Audit cumulative transaction drift

Connected topic: Floating-point arithmetic (§4.3)

Task 09 · audit_drift(transactions)

Try first: Compute the three totals from one normalized amount list, then track the first zero-based index where running float error exceeds one cent.

Answers: see "Task 9: Audit cumulative transaction drift" under Answers at the end.

Task 10: Generate a summation stress case

Connected topic: Floating-point arithmetic (§4.3)

Task 10 · generate_adversarial_case(n)

Try first: Place small terms between a large positive value and its cancelling negative, then handle short lengths.

Answers: see "Task 10: Generate a summation stress case" under Answers at the end.

Made by Thanate Amornjiaporn (Focus)

Answers

Task 1: IEEE 754 bit fields

Task 01 · to_ieee754_bits(x) · worked solution

1. The function returns three fields separated by spaces: one sign bit, eleven exponent bits, and 52 fraction bits. The stored exponent uses bias 1023; the leading 1 of a normal number is implicit and is not part of the fraction field.
2. For x = 13.25, the binary value is 1101.01 (base 2) = 1.10101 (base 2) × 2^3. The sign is 0, the stored exponent is 3 + 1023 = 1026 = 10000000010 (base 2), and the fraction begins 10101 followed by zeroes to make 52 bits.
3. Convert the Python float to its raw 64 bits with struct.pack, then int.from_bytes(..., "big") and format(..., "064b"). Slice the 64 characters at 1 and 12; those are the sign/exponent and exponent/fraction boundaries.
4. Check that the three returned field lengths are 1, 11, and 52 and that concatenating them gives exactly 64 bits. The fraction starts after the leading normalized 1; do not include that hidden bit.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
def to_ieee754_bits(x):
  raw = struct.pack("!d", x)
  bits = f"{int.from_bytes(raw, 'big'):064b}"
  sign = bits[:1]
  exponent = bits[1:12]
  mantissa = bits[12:]
  return f"{sign} {exponent} {mantissa}"
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Made by '1

Task 2: Classify IEEE 754 values

Task 02 · classify_float(x) · worked solution

1. Use the sign/exponent/fraction fields from Task 1. The sign does not change whether a value is zero, subnormal, normal, infinity or NaN.
2. If all 11 exponent bits are zero, an all-zero fraction means zero and a nonzero fraction means subnormal. If all exponent bits are one, an all-zero fraction means infinity and a nonzero fraction means NaN. Every other exponent pattern is normal.
3. This works for negative zero and negative infinity too because classification depends on exponent and fraction. No float equality or math.isnan/math.isinf calls are needed.
4. Check representatives: 0.0 → zero; the smallest positive subnormal → subnormal; 1.0 → normal; positive infinity → inf; and a NaN → nan. A negative zero must still classify as zero.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
def classify_float(x):
  sign, exponent, fraction = to_ieee754_bits(x).split()
  if exponent == "0" * 11:
      return "zero" if fraction == "0" * 52 else "subnormal"
  if exponent == "1" * 11:
      return "inf" if fraction == "0" * 52 else "nan"
  return "normal"
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Made by Thanate

Task 3: Count representable floats between two values

Task 03 · ulp_distance(a, b) · worked solution

1. Raw IEEE bit patterns do not sort numerically across the sign boundary: negative values have the sign bit set and their magnitude ordering runs backward. First map each pattern to a monotonic integer key.
2. For a negative pattern, invert every bit and add one; for a nonnegative pattern, set the sign bit. The added one maps negative zero onto positive zero, so the two encodings share one numeric key. Then the absolute difference counts representable steps. Treat NaNs as unordered and define the behavior explicitly; this solution raises ValueError for them.
3. If a and b compare equal, return zero immediately. That includes +0.0 and -0.0, which are two encodings of the same numeric value. Infinities remain valid endpoints in the ordering.
4. Check adjacent values with math.nextafter in both the positive and negative regions: their distance should be one. Check symmetry by swapping the arguments. Do not subtract the floats themselves; that measures numeric separation, not ULP steps.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
def _ordered_key(x):
  if classify_float(x) == "nan":
      raise ValueError("ULP distance is undefined for NaN")
  raw = int(to_ieee754_bits(x).replace(" ", ""), 2)
  sign = 1 << 63
  mask = (1 << 64) - 1
  return (( raw & mask) + 1) if raw & sign else (raw | sign)

def ulp_distance(a, b):
  if a == b: # includes +0.0 and -0.0
      return 0
  return abs(_ordered_key(a) - _ordered_key(b))
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Made By

Task 4: Compare ordinary and compensated sums

Task 04 · naive_sum(values) · worked solution

1. A floating-point addition aligns exponents, adds significands, and rounds the result back to the available precision. In binary64, a value that is too small to affect a much larger running total can disappear during that rounding.
2. The naive version performs the specified left-to-right accumulation from 0.0. Its answer can depend on order; cancellation can expose rounding errors introduced earlier.
3. For an illustrative sequence [1e16, 1.0, 1.0, -1e16], each 1.0 is below half an ULP at 1e16, so the naive running total remains 1e16 until the final cancellation and returns 0.0. The exact sum is 2.0.
4. Use this small example to validate behavior, but calculate the exact drift for your own assigned dataset separately. The assignment asks for the naive-versus-Kahan difference in cents: compute ‘100 × abs(naive_total − kahan_total)‘ because the transaction amounts are in dollars, and report the full result in cents with the input order. For the exact difference between the two stored float results, subtract their ‘Decimal.from_float(...)‘ values before multiplying by 100; do not round it to a few guessed decimal places.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
def naive_sum(values):
  total = 0.0
  for value in values:
      total += value
  return total
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Made by Tha

Task 5: Implement Kahan compensated summation

Task 05 · kahan_sum(values) · worked solution

1. Kahan summation keeps a running total and a compensation for low-order bits lost in the previous addition. The correction is not an extra exact accumulator; it recovers some information that ordinary addition rounded away.
2. For each value, subtract the saved correction to form adjusted. Add adjusted to the total, then estimate the new lost part as (updated - total) - adjusted. Save that amount for the next iteration.
3. The sign in this correction formula matters. Use the consistent four-line recurrence shown below; mixing it with a variant that stores the opposite sign breaks the compensation.
4. Check the empty input returns 0.0 and compare the adversarial example from Task 4 with naive_sum. Kahan often improves accuracy, but it does not guarantee the correctly rounded exact sum for every sequence.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
def kahan_sum(values):
  total = 0.0
  correction = 0.0
  for value in values:
      adjusted = value - correction
      updated = total + adjusted
      correction = (updated - total) - adjusted
      total = updated
  return total
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Made by Thai

Task 6: Detect likely catastrophic cancellation

Task 06 · detect_catastrophic_cancellation(a, b) · worked solution

1. When two large, nearly equal values are subtracted, leading digits cancel and the result may have far fewer reliable significant digits than either input. The assignment defines a practical heuristic: |a-b| ≤ max(|a|,|b|)·2^-52, provided the scale is nonzero.
2. First reject NaN and infinity so expressions such as infinity minus infinity do not produce misleading comparisons. Then compute the scale and apply the stated threshold exactly.
3. For a = 10^16 and b = 10^16-2, the scale is 10^16 and the threshold is about 2.22. The difference is 2, so the heuristic returns True.
4. This Boolean test flags a risk; it does not measure the actual error or prove the answer is inaccurate. The subtraction of two nearby stored floats may itself be exact, even though both operands already carry rounding error from earlier calculations.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
def detect_catastrophic_cancellation(a, b):
  if classify_float(a) not in {"zero", "subnormal", "normal"}:
      return False
  if classify_float(b) not in {"zero", "subnormal", "normal"}:
      return False
  scale = max(abs(a), abs(b))
  return scale != 0 and abs(a - b) <= scale * 2**-52
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Made by Thanat

Task 7: Compare floats with a relative tolerance

Task 07 · safe_float_equal(a, b, tol=1e-9) · worked solution

1. Exact equality is usually too strict for computed results because rounding creates small differences. A relative comparison accepts a difference no larger than tol times the larger operand magnitude.
2. NaN is unequal to every value, including itself. Infinities compare equal only when both are infinities with the same sign; finite arithmetic should not be used to compare them.
3. For finite values, let scale = max(|a|,|b|). If scale is zero, both values are zero and the difference is zero. Otherwise check |a-b| $\leq$ tol·scale. This is a relative-only rule; near zero it intentionally allows almost no absolute error.
4. Check symmetry by swapping a and b. If the application also needs an absolute tolerance near zero, the specification must say so; do not silently change this task's stated relative tolerance.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
def safe_float_equal(a, b, tol=1e-9):
  kind_a = classify_float(a)
  kind_b = classify_float(b)
  if kind_a == "nan" or kind_b == "nan":
      return False
  if kind_a == "inf" or kind_b == "inf":
      return kind_a == kind_b and ((a > 0) == (b > 0))
  scale = max(abs(a), abs(b))
  if scale == 0:
      return True
  return abs(a - b) <= tol * scale
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Made by Tha

Task 8: Represent money as integer cents

Task 08 · FixedPointCents · worked solution

1. Binary floating point cannot represent most decimal fractions exactly. A currency quantity restricted to cents can instead be stored as an integer number of cents; addition and subtraction then use exact integer arithmetic.
2. Initialize the object from an integer cent count. Addition and subtraction create a new FixedPointCents object rather than changing either operand. Equality should compare both type and cent value.
3. For the representation, use abs(cents) with divmod(...,100), then attach the sign and print exactly two digits after the decimal point. This also handles negative amounts such as -5 cents as -$0.05.
4. Check 1234 reprs as $12.34, -5 reprs as -$0.05, and 125 cents + (-25 cents) gives 100 cents. Convert decimal input into cents under an explicit rounding policy; do not multiply arbitrary floats and assume the result is exact.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
class FixedPointCents:
  def __init__(self, cents):
      self.cents = int(cents)

  def __add__(self, other):
      if not isinstance(other, FixedPointCents):
          return NotImplemented
      return FixedPointCents(self.cents + other.cents)

  def __sub__(self, other):
      if not isinstance(other, FixedPointCents):
          return NotImplemented
      return FixedPointCents(self.cents - other.cents)

  def __eq__(self, other):
      return (isinstance(other, FixedPointCents)
              and self.cents == other.cents)

  def __repr__(self):
      sign = "-" if self.cents < 0 else ""
      dollars, cents = divmod(abs(self.cents), 100)
      return f"{sign}${dollars}.{cents:02d}"
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Task 9: Audit cumulative transaction drift

Task 09 · audit_drift(transactions) · worked solution

1. Normalize each input record first: the sheet allows dictionaries with an amount field or raw numeric amounts. Use that same ordered list for naive_sum, kahan_sum and the running comparison.
2. Convert each stated decimal amount to integer cents using Decimal(str(amount)) and an explicit rounding rule (here, half-even to two decimal places). This prevents the conversion itself from adding a binary-float artifact to the exact ledger total.
3. After each running float addition, compare its exact rational value with the integer-cent total. ‘as_integer_ratio()‘ gives numerator/denominator for the stored float. Multiply the numerator by 100 to express it in cents; error exceeds one cent precisely when the resulting numerator difference is greater than the denominator.
4. Return the totals and the first zero-based index, or None if the threshold is never crossed. Validate on a tiny hand-calculated list, then run your own assigned CSV. The written justification must report your dataset’s actual drift; do not reuse another learner’s result.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
from decimal import Decimal, ROUND_HALF_EVEN

def audit_drift(transactions):
  amounts = [item["amount"] if isinstance(item, dict) else item
             for item in transactions]
  naive_total = naive_sum(amounts)
  kahan_total = kahan_sum(amounts)
  fixed = FixedPointCents(0)
  running = 0.0
  first_index = None
  for index, amount in enumerate(amounts):
      running += amount
      cents = int((Decimal(str(amount)) * 100).quantize(
          Decimal("1"), rounding=ROUND_HALF_EVEN))
      fixed += FixedPointCents(cents)
      numerator, denominator = running.as_integer_ratio()
      error_num = abs(numerator * 100 - fixed.cents * denominator)
      if first_index is None and error_num > denominator:
          first_index = index
  return {"naive_total": naive_total,
          "kahan_total": kahan_total,
          "fixed_total_cents": fixed.cents,
          "first_index_error_exceeds_cent": first_index}
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Task 10: Generate a summation stress case

Task 10 · generate_adversarial_case(n) · worked solution

1. Choose a large number whose ULP is larger than 1.0, such as 10^16 in binary64. Adding 1.0 repeatedly to that running total loses those small terms because they are below half an ULP.
2. Place -10^16 last. The large terms cancel, while the naive sum has already discarded the intervening ones. For $n \geq 3$ this classic sequence exposes order-dependent drift; for $n = 0$ or 1 return an empty or zero-valued list, and for $n = 2$ the pair cancels.
3. Reject negative n because a requested list length cannot be negative. The construction is a clear stress case, not a proof of the mathematically largest possible error for every n or every summation algorithm.
4. Check lengths 0, 1, 2, 3, and a larger n. For $n \geq 2$, the exact sum is n-2; for $n = 0$ or 1, the returned zero-valued list sums to zero. Confirm naive_sum and kahan_sum differ for a suitable n.
5. **Implementation pattern:** place this logic in the required function or class from the exercise scaffold.

```
def generate_adversarial_case(n):
  if n < 0:
      raise ValueError("n must be nonnegative")
  if n < 2:
      return [0.0] * n
  return [1e16] + [1.0] * (n - 2) + [-1e16]
```

6. Before moving on, check the input boundaries and one hand-computed case. Keep the exact required name and return shape from the scaffold.

Made by Thanat

Personal worked solutions based on the APDR Computer Exercise 2 sheet, explained in my own words. The dataset-specific drift and first-crossing results are intentionally left for each learner to calculate from their own assigned transactions. Not an official FAU publication; check course details against current materials. Made by **Thanate Amornjiraporn (Focus)**.